# CHAPTER 3 – PART 1
# SOLVING PROBLEMS BY SEARCHING

## INTRODUCTION TO ARTIFICIAL INTELLIGENT (AI)

REV: 03.03.2024

# Solving Problems by Searching

## INTRODUCTION TO ARTIFICIAL INTELLIGENT

- Goal-based and utility-based agents consider future actions and require searching and planning to achieve their goals.

- In this course, we will focus on a type of goal-based agent known as "a problem-solving agent" that uses searching techniques.

- For simplicity, we will initially limit the environment as:
  - fully observable
  - deterministic
  - single agent
  - static
  - discrete

  The environment may be sequential.

# Solving Problems by Searching

## INTRODUCTION TO ARTIFICIAL INTELLIGENT

- For now, we will consider the solution to a problem as a sequence of actions.

The solution to the problem → A sequence of actions

- The process of looking for a sequence of actions to achieve the goal is called search.

- During the execution of a solution (a sequence of actions), an agent ignores its percepts when choosing an action. Because it knows what they will be in the future. However, if the environment is nondeterministic or multi-agent, it should may need to reassess the percepts at each time step in the future.

# Defining a Problem

A problem is formally defined by the following components:

- **States:** the set of all possible states

$$S = \{S_0, S_1, S_2, \dots\}$$

- **The Initial State:** the state in which the agent starts $(S_0)$

- **Actions:** the possible actions that the agent can do

$$a = \{a_1, a_2, a_3, \dots\}$$

- **The Transition Model:** an exact description of what each action does

When we perform each action $a_k$ on each state $S_i$ , the next (resulting) state should be described.

$$\text{The current state } S_i \xrightarrow{\text{an action } a_k} \text{The next state } S_j$$

This process is called a step.

- **The Goal Test:** Determines whether a given state is a goal state or not.

$$S_i \stackrel{?}{=} \text{Goal State}$$

- **The Step Cost:** The step cost is a function that assigns a numeric cost to each step.

A step cost of performing an action $a_k$ on state $S_i$ to reach state $S_j$. $\Rightarrow$ $c(S_i, a_k, S_j)$

➢ **The Path Cost:** A path cost is the sum of all step costs along a path.

$$\underbrace{S_n \xrightarrow{a_i} S_{n+1} \xrightarrow{a_j} \cdots \xrightarrow{a_t} S_k}_{\text{a path}}$$

its path cost

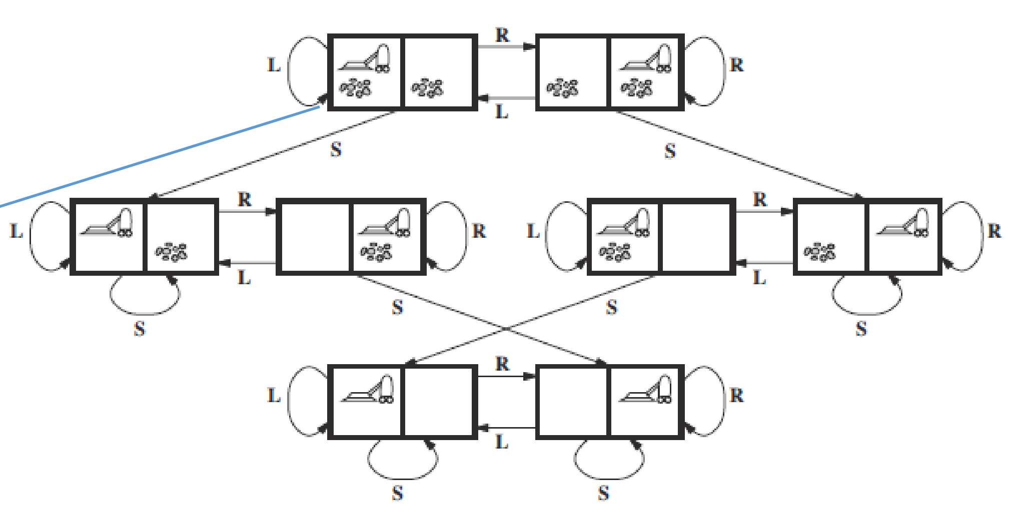
$$c(S_n, a_i, S_{n+1}) + c\left(S_{n+1}, a_j, S_{n+2}\right) + \ldots + c(S_{k-1}, a_t, S_k)$$

A problem-solving agent selects a cost function that reflects its performance measure. The optimal solution can then be determined based on the path costs of solution paths.

$$\underbrace{S_0 \xrightarrow{a_i} S_{n+1} \xrightarrow{a_j} \cdots \text{Goal State}}_{\text{a solution path}}$$

# **Ex:** The Vacuum World

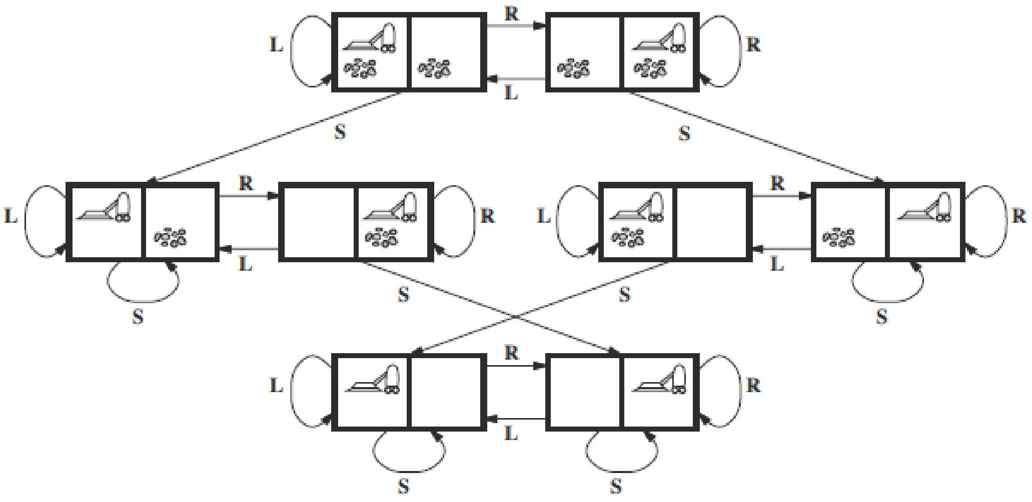
$S_1 = [in\ A, A\ is\ dirty, B\ is\ dirty]$




The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- **States:** The states are determined by both the agent's location and the dirt locations. The agent can be in one of two locations, each of which may or may not contain dirt. Therfore, there are 2 × 2^2 = 8 possible world states.

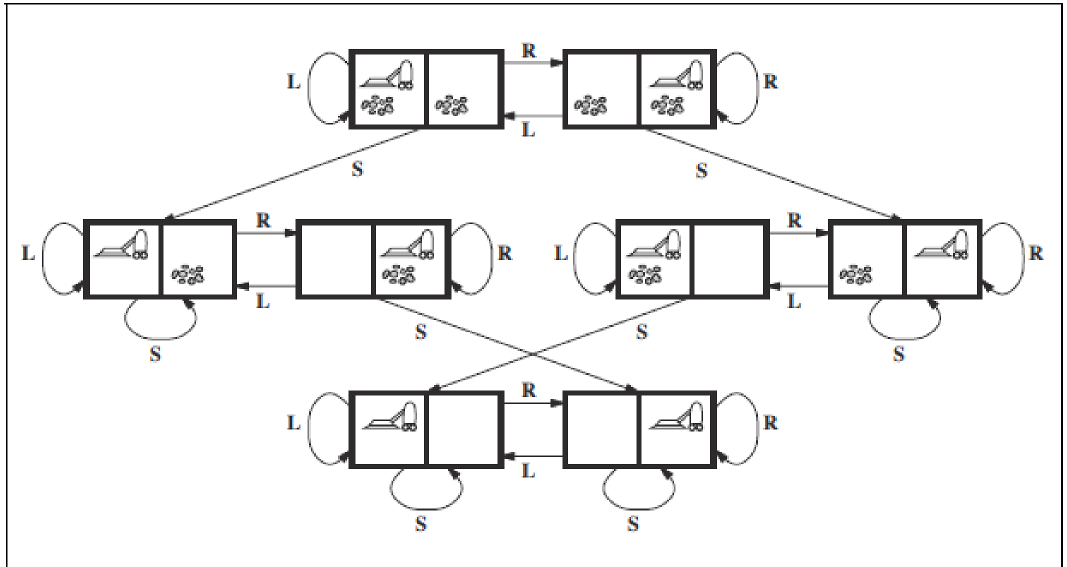
# **Ex:** The Vacuum World




The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- **The Initial State:** Any state can be determined as the initial state.
- **Actions:** Each state has three actions: $a_1 = left$, $a_2 = right$, and $a_3 = suck$.

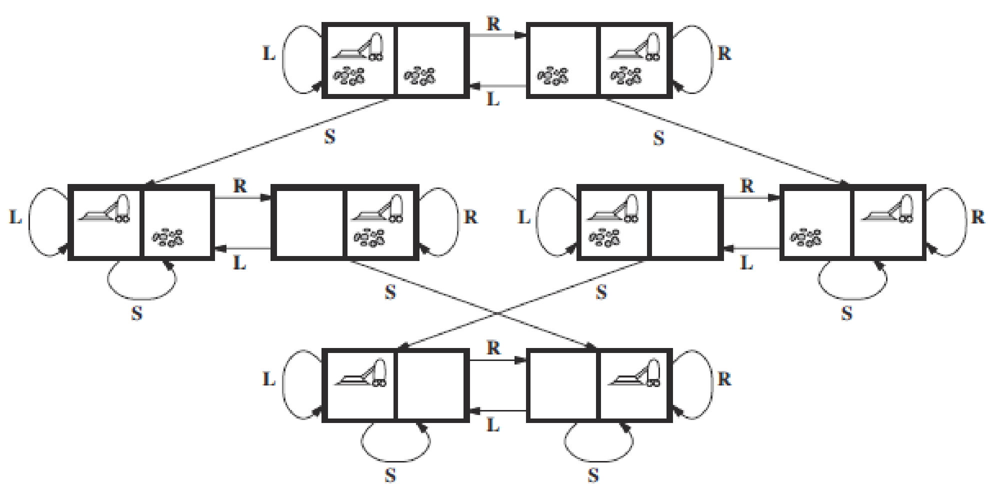
# **Ex:** The Vacuum World




The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- **The Transition Model:** The actions have their expected effects, except that moving left in the left square, moving right in the right square, and sucking in a clean square has no effect.

# **Ex:** The Vacuum World

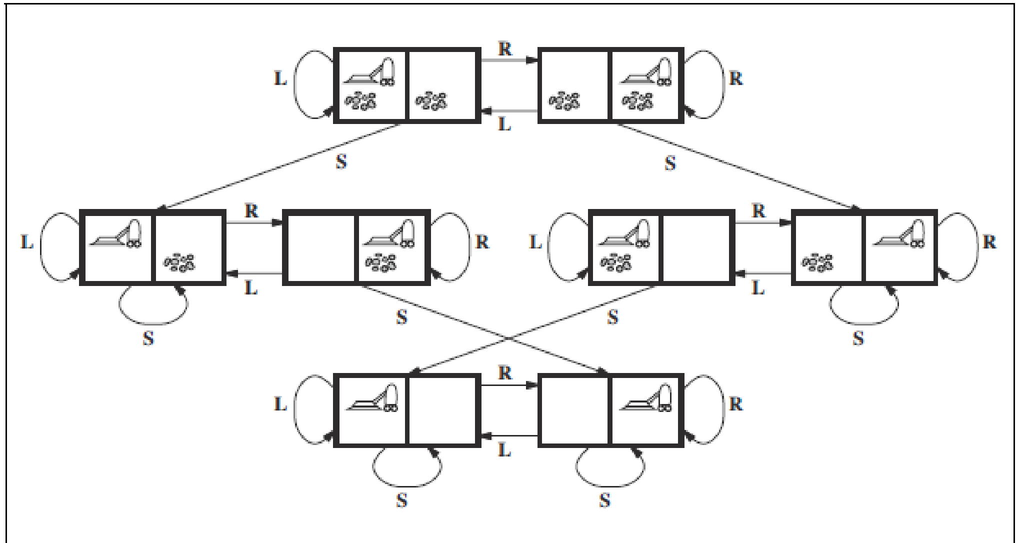



The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- **Goal Test:** It checks whether all the squares are clean.

$$S_n \stackrel{?}{=} [in\ A, A\ is\ clean, B\ is\ clean] \quad \text{or} \quad S_n \stackrel{?}{=} [in\ B, A\ is\ clean, B\ is\ clean]$$

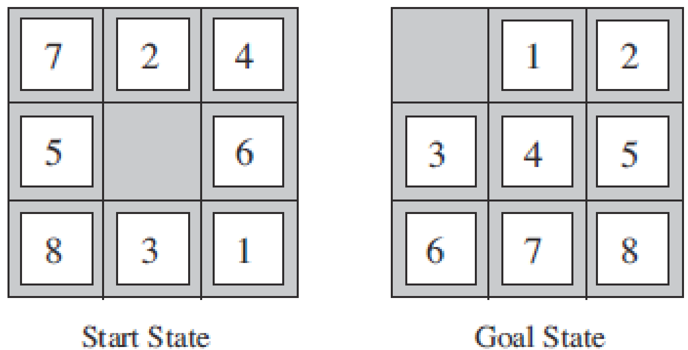
# **Ex:** The Vacuum World




The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- **Step Cost:** Each step costs 1. $c(S_i, a_k, S_j) = 1 \text{ for all } i, k$
- **Path Cost:** The path cost is the number of steps in the path.

# **Ex:** The 8-puzzle



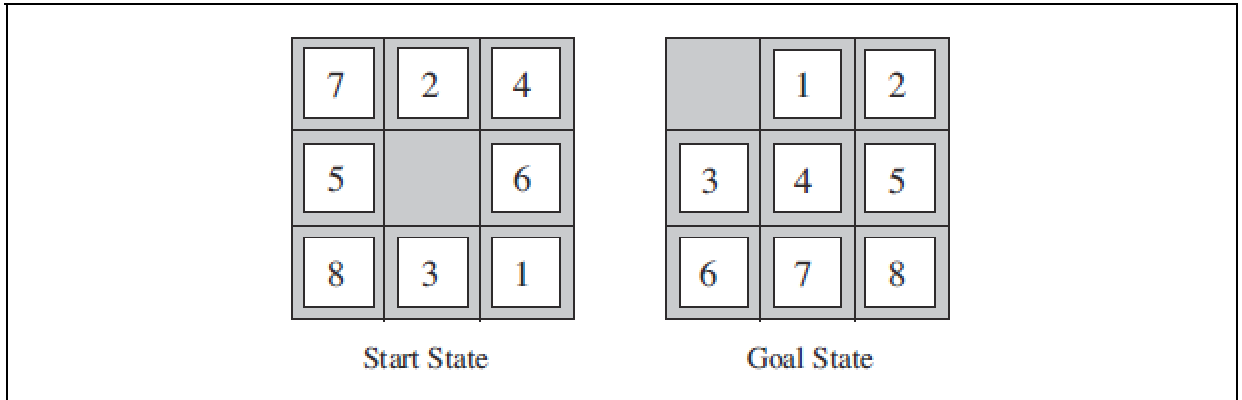

A typical instance of the 8-puzzle.

- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares. The 8-puzzle has 9!/2=181.440 reachable (solvable) states [1].

- **The Initial State:** Any state can be designated as the initial state.

[1] Half of the initial states for the *n*-puzzle problem are impossible to resolve, no matter how many moves are made. See for the proof: Johnson, Wm. Woolsey; Story, William E. (1879), "Notes on the "15" Puzzle", American Journal of Mathematics, 2 (4): 397–404

# **Ex:** The 8-puzzle

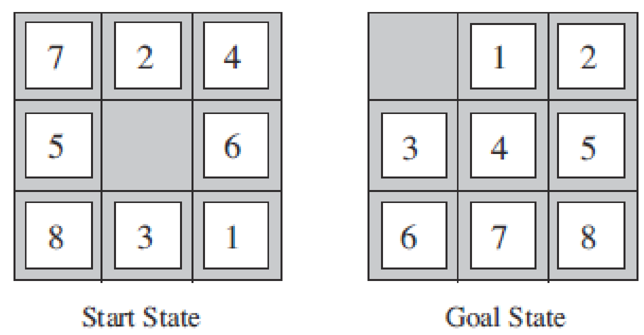



A typical instance of the 8-puzzle.

- **Actions:** The simplest formulation defines the actions as movements of the blank space $a_1 = left$, $a_2 = right$, $a_3 = up$, or $a_4 = down$. Different subsets of these are possible depending on where the blank is.

- **The Transition Model:** Given a state and action, this returns the resulting state: For example, if we apply left to the start state in the above figure, the resulting state has the ‘5’ and the blank swapped.

# **Ex:** The 8-puzzle



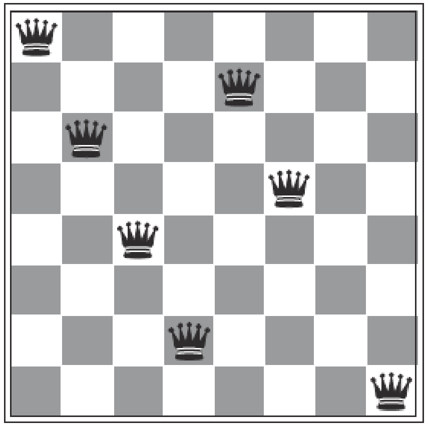

A typical instance of the 8-puzzle.

- **Goal test:** It checks whether the state matches the goal configuration shown in the figure. (Other goal configurations are possible.)

- **Step Cost:** Each step costs 1.

- **Path Cost:** The path cost is the number of steps in the path.

# **Ex:** The 8-Queens Problem



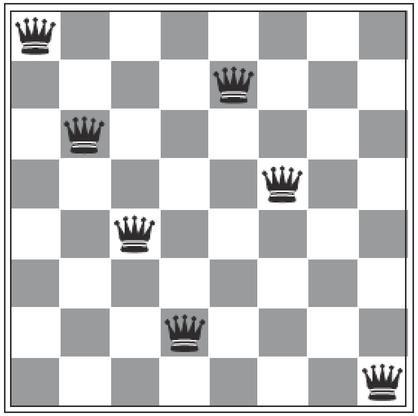
Almost a solution to the 8-queens problem. (Solution is left as an exercise.)

- **States:** Any arrangement of 0 to 8 queens on the board is a state. The number of possible states is $64 \times 63 \times 62 \times \ldots \times 57 \approx 1.8 \times 10^{14}$.

- **The Initial State:** No queens on the board.

- **Actions:** Add a queen to an empty square. For ex: $a_1 = put(A1), a_2 = put(A2), \ldots$

# **Ex:** The 8-Queens Problem



Almost a solution to the 8-queens problem. (Solution is left as an exercise.)

- **The Transition Model:** Returns the board with a queen added to the specified square
- **Goal test:** 8 queens are on the board, and none attacked.
- **Step Cost:** Each step costs 1.
- **Path Cost:** Each solution contains the same path costs, 8.

# **Ex:** Knuth's Problem

Donald Knuth claim that any positive integer can be obtained by applying a sequence of factorial, square root operations, and floor operations on '4'. For example, we can obtain '5' from '4' as follows:

$$\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \right\rfloor = 5$$

- **States:** Positive numbers (real or integer). Infinite numbers of states.
- **The Initial State:** 4
- **Actions:** $a_1 = apply\_square\_root$, $a_2 = floor$, $a_3 = factorial$ (factorial for integers only).
- **The Transition Model:** As given by the mathematical definitions of the operations.

# **Ex:** Knuth's Problem

Donald Knuth claim that any desired positive integer can be obtained by applying a sequence of factorial, square root operations, and finally floor operations on '4'. For example, we can obtain '5' from '4' as follows:

$$\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \right\rfloor = 5$$

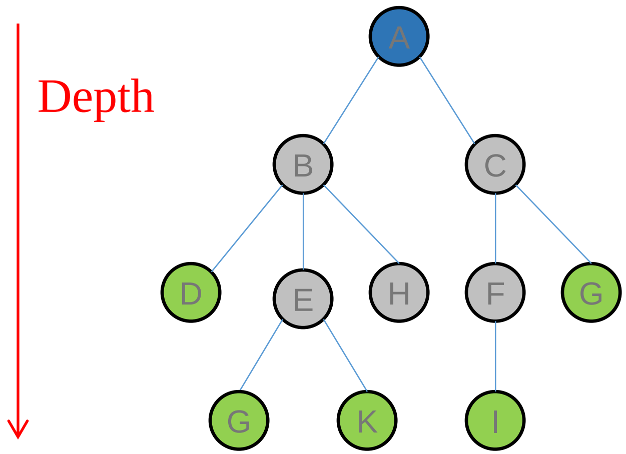
- **Goal test:** State is the desired positive integer.
- **Step Cost:** Each step costs 1.
- **Path Cost:** The path cost is the sum of path costs (the sum of operations performed).

# Consider the following problems:

- Sudoku
- Route finding
- Cargo transportation
- Public transportation in Istanbul
- Automatic assembly sequencing
- …

# Tree Diagrams

The possible paths that a problem-solving agent can be traced can be illustrated by tree diagrams.

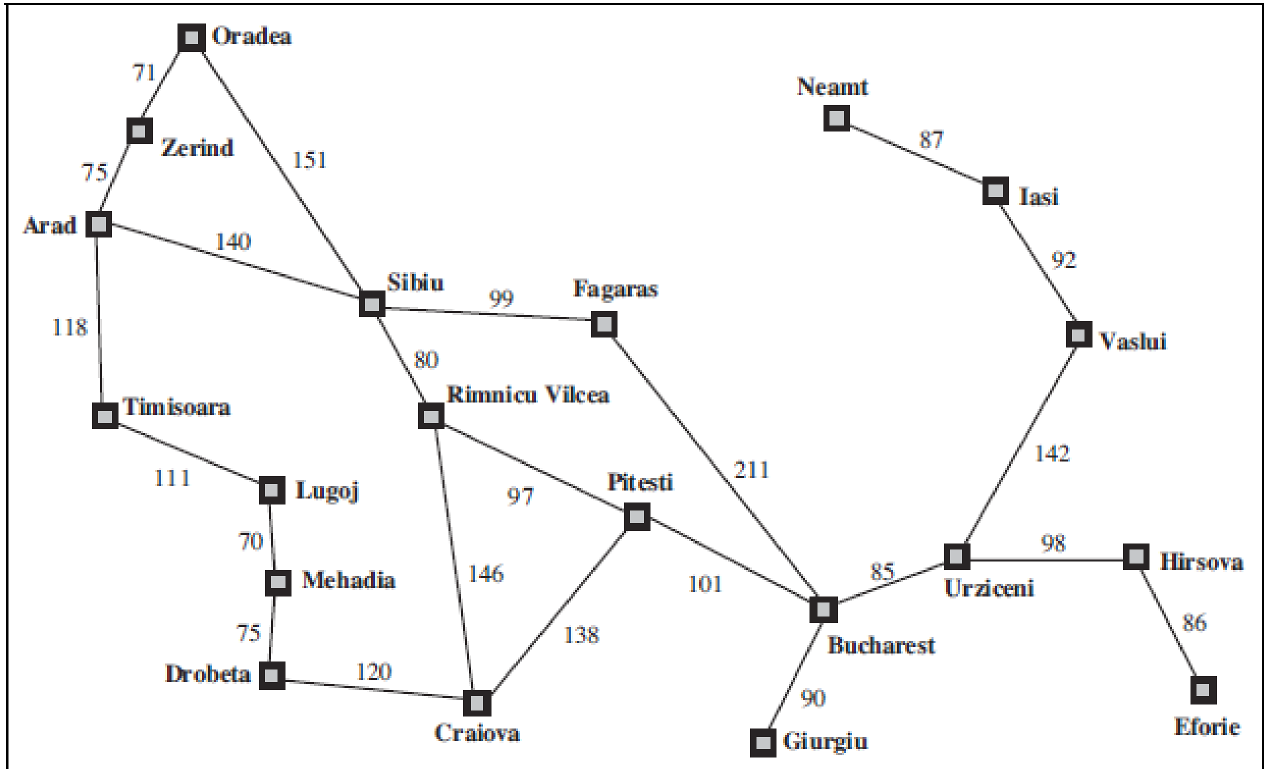



- Each node represents a state. Sometimes different nodes can represent the same state.
- The branches are actions that the agent does.
- The set of all leaf nodes available for expansion at any given instant is called the frontier (open list).

The root

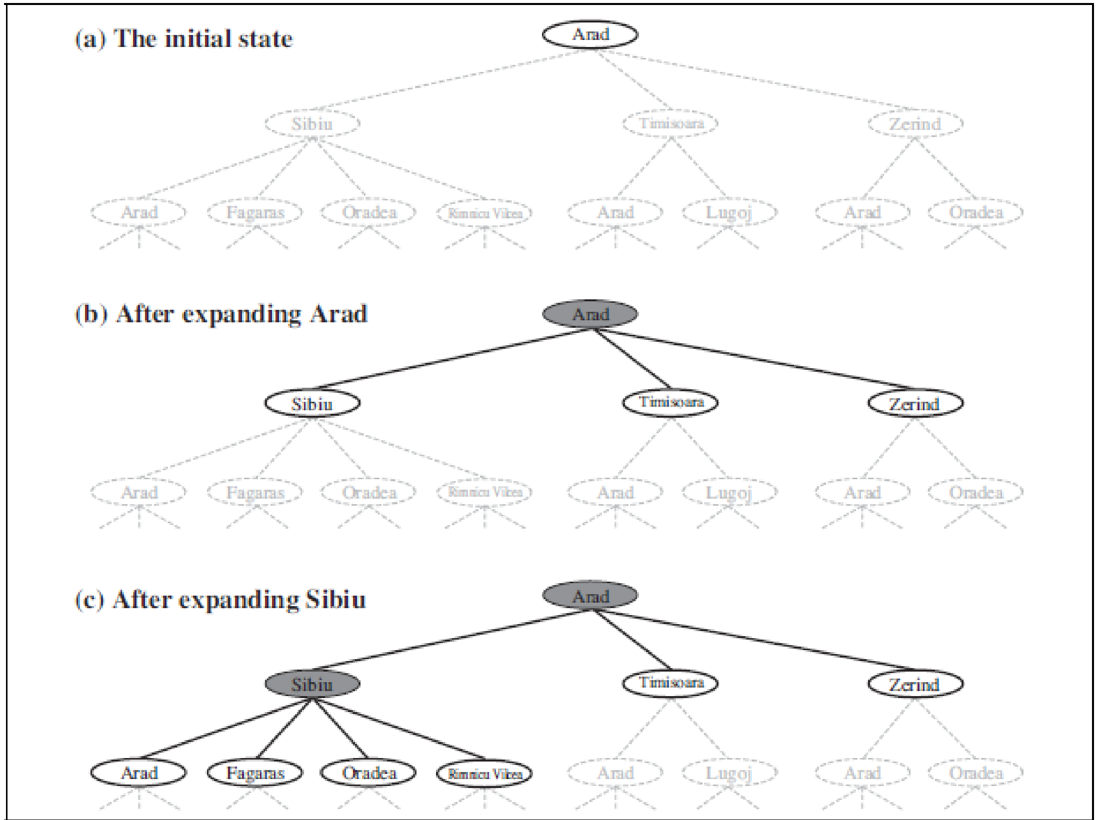
Leaf nodes: They have no children in the tree.

# **Ex:** The Map of Romania



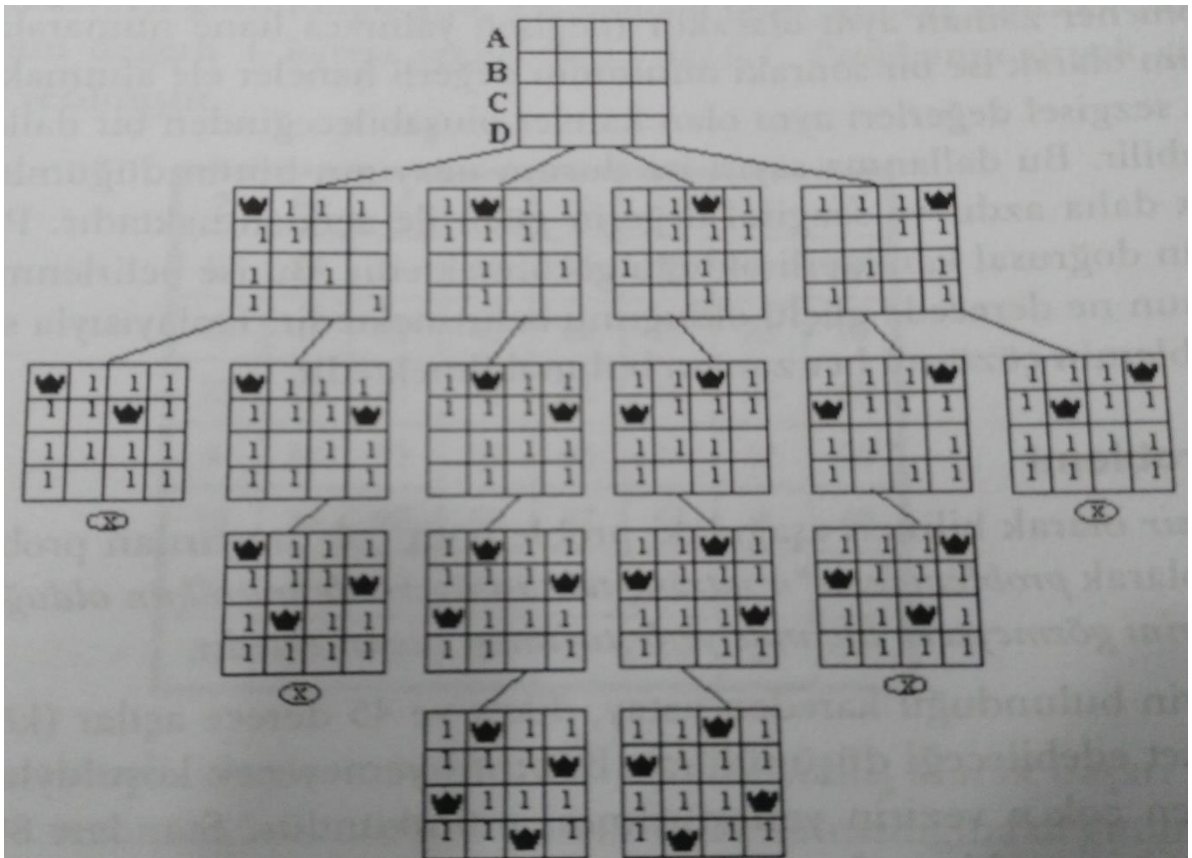

A simplified road map of part of Romania.

# **Ex:** The Map of Romania




Partial search trees for finding a route from Arad to Bucharest. Nodes that have been expanded are shaded; nodes that have been generated but not yet expanded are outlined in bold; nodes that have not yet been generated are shown in faint dashed lines.

# Ex : The 4-Queens Problem



# Solving Problems by Searching

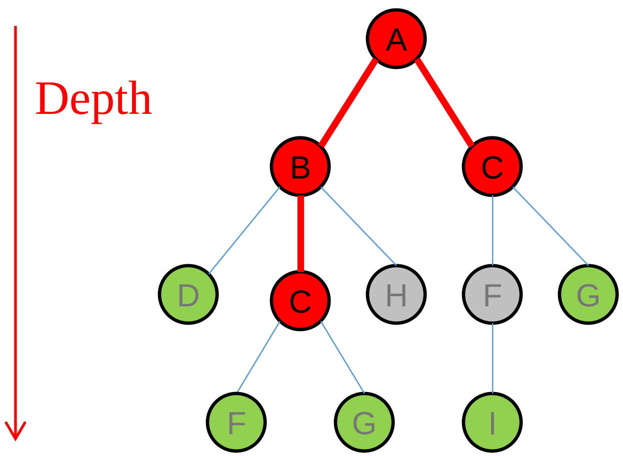
## INTRODUCTION TO ARTIFICIAL INTELLIGENT

function TREE-SEARCH(problem) returns a solution or failure

- initialize the frontier using the initial state of the problem
- loop do
  - if the frontier is empty, then return failure
  - choose a leaf node and remove it from the frontier
  - if the node contains a goal state, then return the corresponding solution
  - expand the chosen node, adding the resulting nodes to the frontier

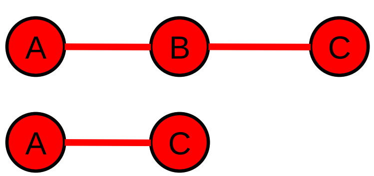
The process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand. Search algorithms have a basic structure named tree-search algorithm given above; they mainly differ in terms of how they select which state to expand next—the so-called search strategy.

# Search Tree



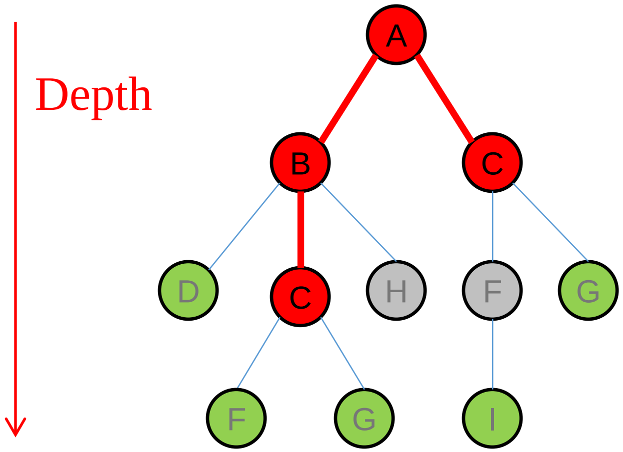

The redundant paths exist whenever there is more than one way to get from one state to another. For example, the example of the map of Romania contains many redundant paths, and you may also get stuck in an infinite loop.

If step cost is 1, the path A-B-C is a redundant path.



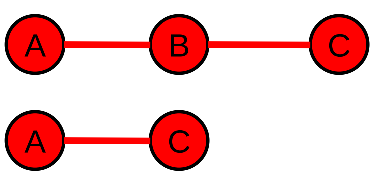

The algorithms that forget their history may get stuck in an infinite loop.

# Search Tree



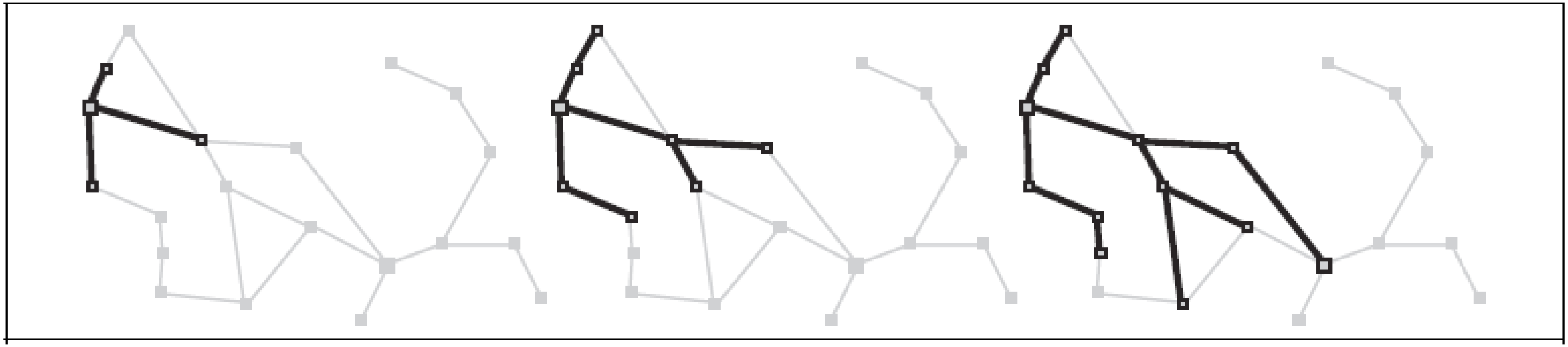

The way to avoid exploring redundant paths is to remember its past. To do this, we improve the tree-search algorithm with a data structure called the explored set (closed list), which remembers every expanded node. Thus, we can eliminate redundant paths.

If step cost is 1, the path A-B-C is a redundant path.



# Solving Problems by Searching

## INTRODUCTION TO ARTIFICIAL INTELLIGENT



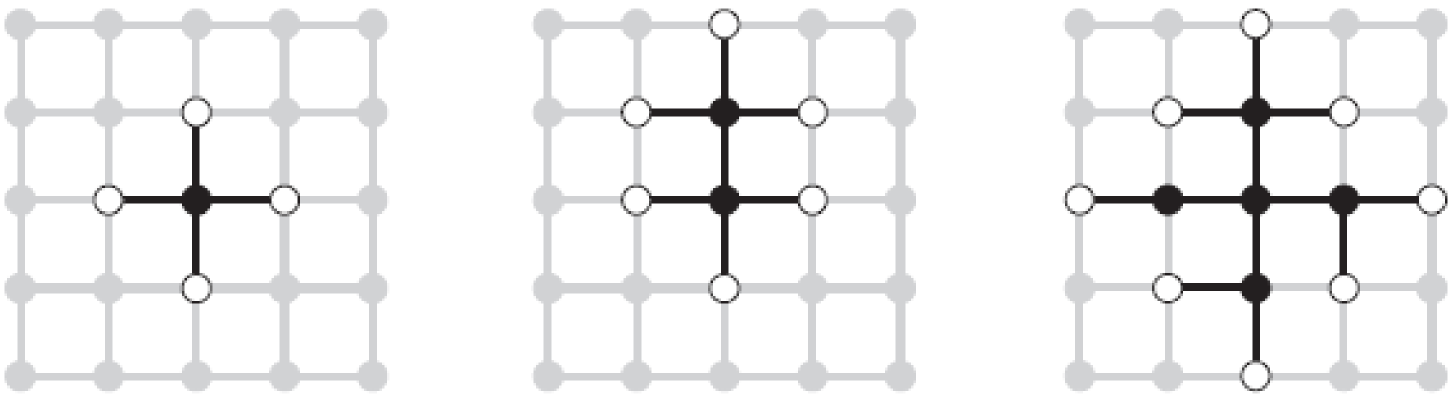
Newly generated nodes that have already been inserted into the explored set or the frontier can be eliminated instead of being added to the frontier. The frontier separates the state-space graph into the explored region and the unexplored region so that every path from the initial state to an unexplored state has to pass through a state in the frontier.

function GRAPH-SEARCH (problem) returns a solution or failure

- initialize the frontier using the initial state of the problem
- initialize the explored set to be empty
- loop do
  - if the frontier is empty, then return failure
  - choose a leaf node and remove it from the frontier
  - if the node contains a goal state, then return the corresponding solution
  - add the node to the explored set
  - expand the chosen node, adding the resulting nodes to the frontier only if they are not in the frontier or explored set

The new algorithm, which is given above, is called the graph-search algorithm.

# **Ex:** A rectangular-grid problem for graph-search



(a) (b) (c)

**Figure 3.9** The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

# Main Reference

Stuart Russell, Peter Norvig; “Artificial Intelligence A Modern Approach”, Pearson, 3rd Edition, 2013.